

Distributed E-Voting System using Microservices & Blockchain

AS2023475 – I.M.Rathnayake

AS2023631 – S.A.C.Hansika

AS2023633 – A.D.N.Methmal



Background & Problem Statement

- **Traditional Voting Issues :**

High cost, paper waste, physical presence needed, and manual counting errors.

- **Centralized E-Voting Risks :**

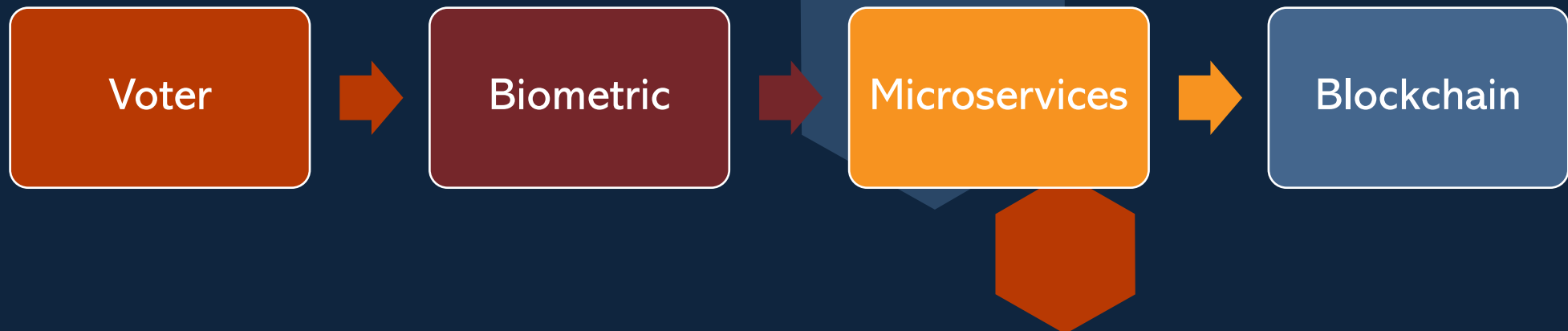
High vulnerability to database manipulation and single points of failure.

- **Weak Authentication :**

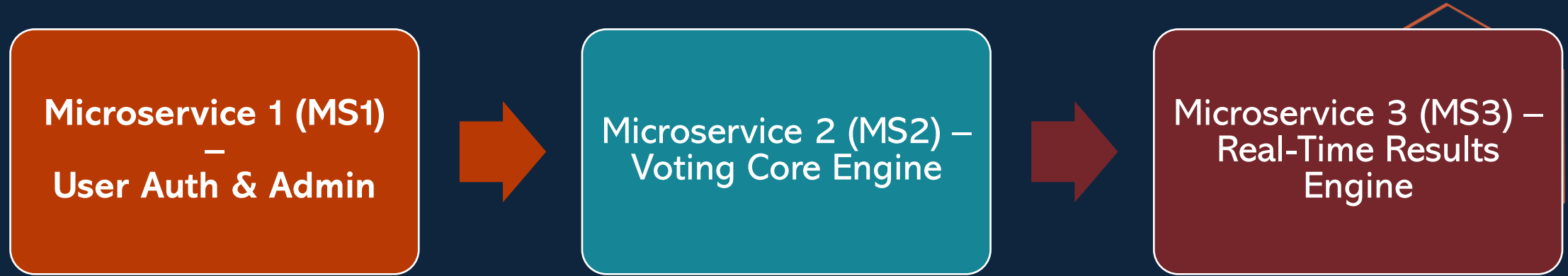
Password or OTP-based voting can easily lead to identity theft.

Proposed Solution

- **Decentralized Architecture** : Uses Microservices and Blockchain technology to eliminate single points of failure.
- **Biometric Security** : Integration of WebAuthn (Fingerprint/FaceID) to prevent identity fraud.
- **Immutable Voting** : Every vote is stored permanently on an Ethereum Blockchain (Ganache).



Overall System Architecture



Containerized Deployment: Entire system runs inside Docker containers using docker-compose.

Microservices Isolation: MS1 (Auth), MS2 (Voting Core), and MS3 (Results Engine) run independently.

Database & Ledger: MongoDB stores user profiles; Ethereum Blockchain stores immutable vote logs.

Microservices Breakdown & Responsibilities

MS1 (User Auth & Admin)

Handles user onboarding, biometric authentication (WebAuthn), JWT token issuance, and election setup.

MS2 (Voting Core Engine)

Verifies JWT tokens, executes Smart Contracts, prevents double-voting, and logs votes to the Ethereum Blockchain.

MS3 (Results Engine)

Reads vote counts directly from the Blockchain smart contract in real-time, completely bypassing MS1 & MS2.

Microservice 1 (MS1) – User Auth & Admin Service

- **Election Management** : Enables the System Administrator to schedule elections, set start/end times, and define voting options.
- **Voter Onboarding** : Processes batch voter registration via CSV file upload and triggers automated email invitations with unique voting links.
- **Passwordless Biometric Auth** : Integrates WebAuthn standard to authenticate voters using hardware biometrics (Fingerprint / FaceID / TouchID).
- **JWT Token Security** : Issues cryptographically signed JSON Web Tokens (JWT) upon successful authentication to grant secure voting access.

Microservice 1 (MS1) – User Auth & Admin Service

Technologies Used (Tech Stack)

- **Backend:** Node.js (v20), Express.js
- **Authentication Standard:** WebAuthn Protocol (@simplewebauthn/server)
- **Security & Tokens:** Web Crypto API, JSON Web Token (JWT)
- **Database:** MongoDB (with Mongoose ORM)
- **Utilities:** Nodemailer (Email service), csv-parser (File handling)
- **Frontend UI:** HTML5, Tailwind CSS, JavaScript (Fetch API)

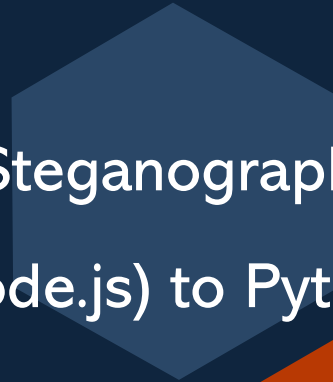
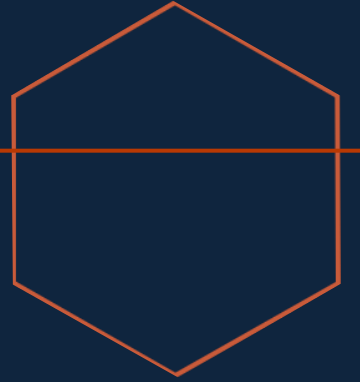
Microservice 2 (MS2) – Voting Core Engine

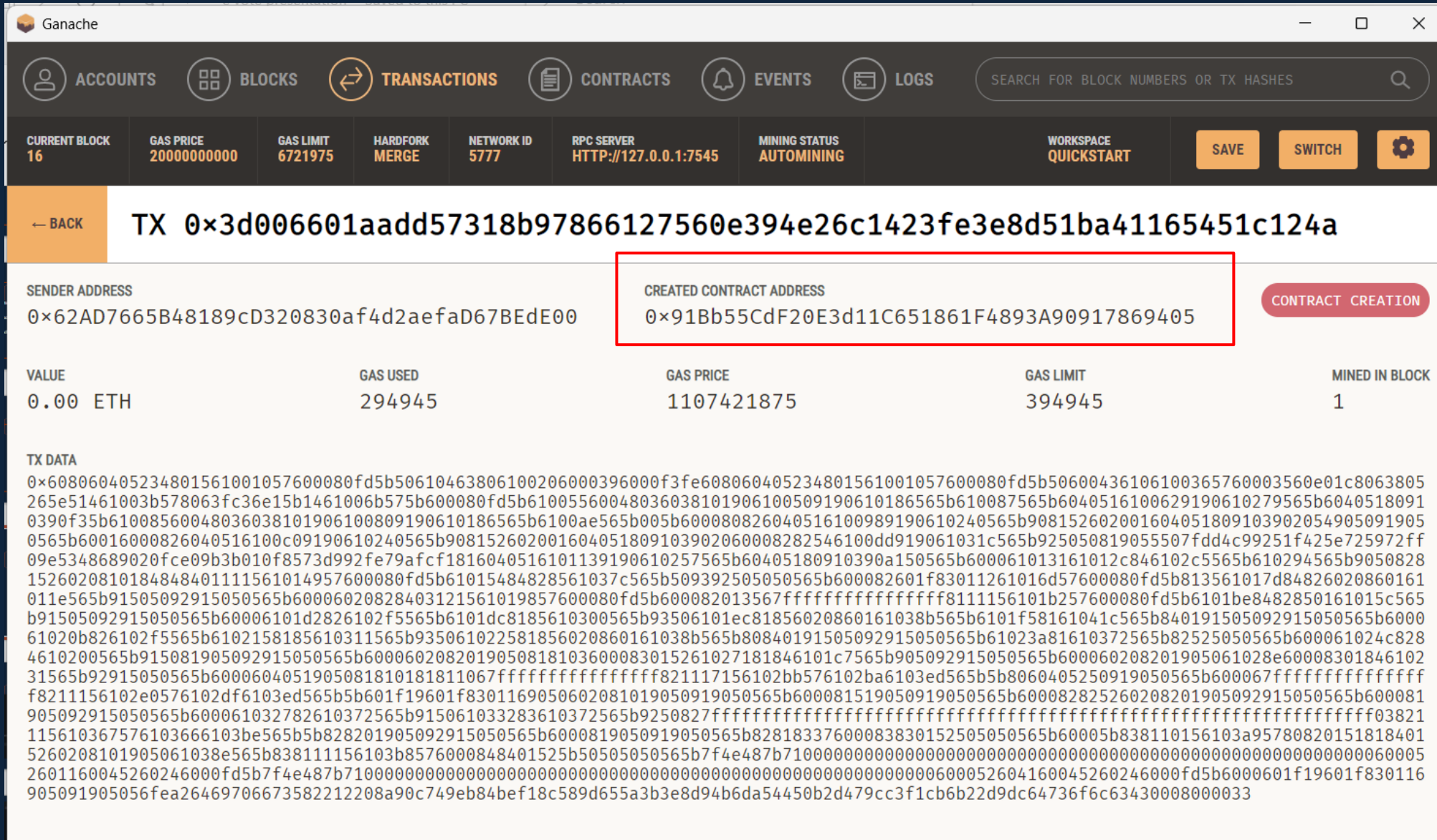
- **JWT Token Validation:** Verifies the cryptographic signature of the voter's JWT token issued by MS1 before allowing any interaction.
- **Smart Contract Execution:** Interacts directly with the Ethereum Blockchain smart contract to record votes as immutable transactions.
- **Double Voting Prevention:** Enforces strict state checks on the Blockchain to reject multiple voting attempts from the same voter identity.

Microservice 2 (MS2) – Voting Core Engine

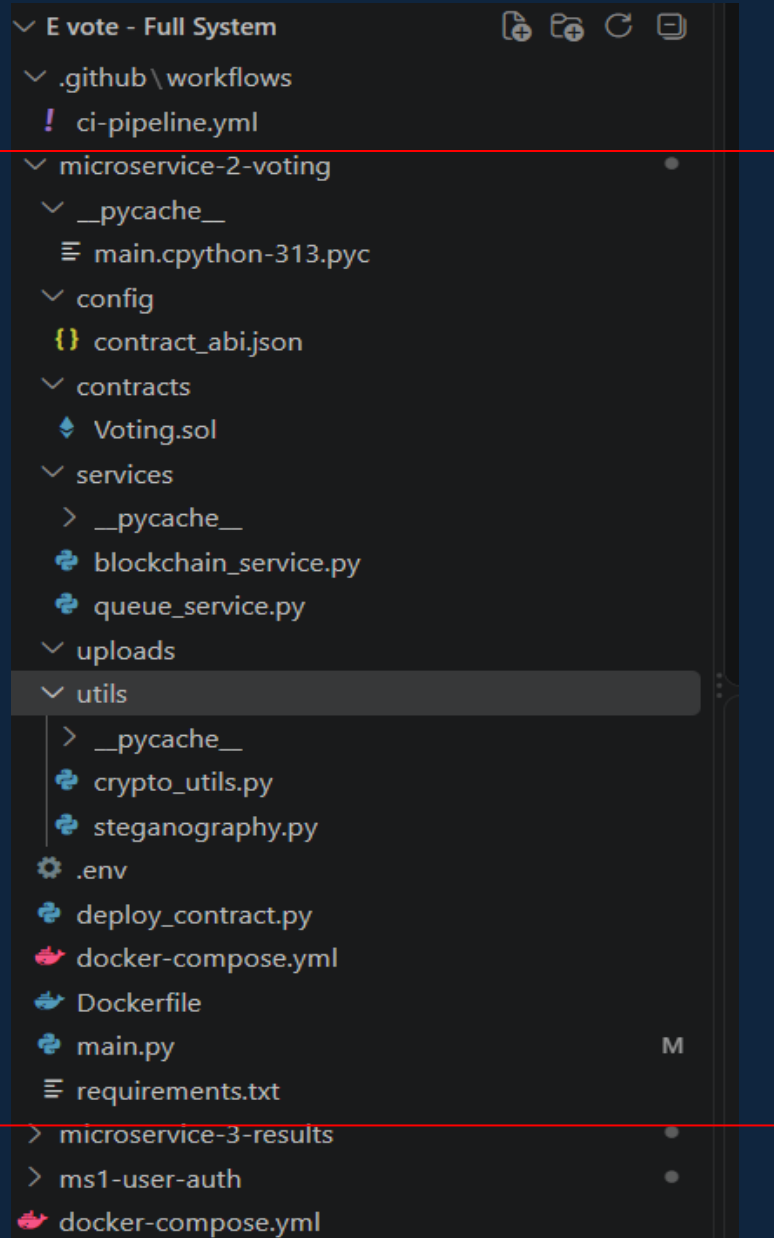
Technologies Used (Tech Stack)

- **Backend:** Python (FastAPI / Flask)
- **Blockchain Network:** Local Ethereum Blockchain (Ganache)
- **Smart Contract Language:** Solidity (Voting.sol)
- **Blockchain Interactivity:** Web3.py
- **Security & Encryption:** Crypto Utilities, Steganography Module
- **Inter-Service Communication:** Axios (Node.js) to Python REST API





Microservice 2 (MS2) – Voting Core Engine

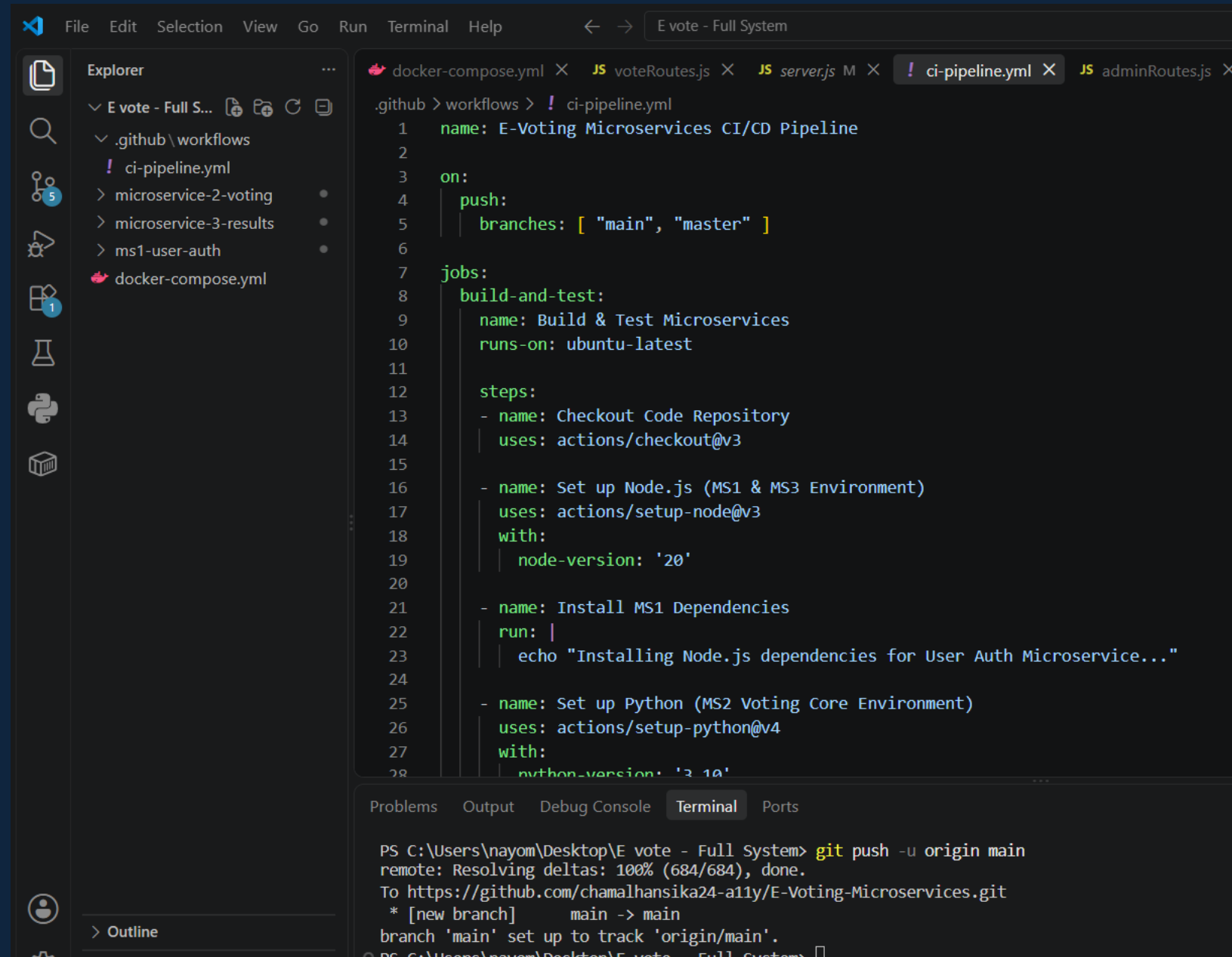


Voting.sol

```
microservice-2-voting > contracts > 🔗 Voting.sol
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.0;
3
4  contract Voting {
5      mapping(string => uint256) private voteCounts;
6
7      event VoteCast(string candidate);
8
9      function vote(string memory candidate) public {
10         voteCounts[candidate] += 1;
11         emit VoteCast(candidate);
12     }
13
14     function getVotes(string memory candidate) public view returns (uint256) {
15         return voteCounts[candidate];
16     }
17 }
```

Automated DevSecOps Pipeline (GitHub Actions)

✓ MS1, MS2, and MS3 Docker
Build & Environment
Verification Passed.



The screenshot displays the Visual Studio Code interface. On the left, the Explorer sidebar shows the file structure of a project named 'E vote - Full System', including a '.github/workflows' directory with a 'ci-pipeline.yml' file. The main editor area shows the content of 'ci-pipeline.yml', which defines a GitHub Actions workflow for building and testing microservices. The workflow includes steps for checking out the repository, setting up Node.js and Python environments, and installing dependencies. The bottom panel shows the Terminal output, which confirms the successful push of the workflow to the GitHub repository.

```
File Edit Selection View Go Run Terminal Help
E vote - Full System

Explorer
E vote - Full S...
  .github\workflows
    ! ci-pipeline.yml
    > microservice-2-voting
    > microservice-3-results
    > ms1-user-auth
    docker-compose.yml

.ci-pipeline.yml
1 name: E-Voting Microservices CI/CD Pipeline
2
3 on:
4   push:
5     branches: [ "main", "master" ]
6
7 jobs:
8   build-and-test:
9     name: Build & Test Microservices
10    runs-on: ubuntu-latest
11
12    steps:
13      - name: Checkout Code Repository
14        uses: actions/checkout@v3
15
16      - name: Set up Node.js (MS1 & MS3 Environment)
17        uses: actions/setup-node@v3
18        with:
19          node-version: '20'
20
21      - name: Install MS1 Dependencies
22        run: |
23          echo "Installing Node.js dependencies for User Auth Microservice..."
24
25      - name: Set up Python (MS2 Voting Core Environment)
26        uses: actions/setup-python@v4
27        with:
28          python-version: '3.10'
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\nayom\Desktop\E vote - Full System> git push -u origin main
remote: Resolving deltas: 100% (684/684), done.
To https://github.com/chamalansika24-a11y/E-Voting-Microservices.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\nayom\Desktop\E vote - Full System>
```

Automated DevSecOps Pipeline (GitHub Actions):

✓ MS1, MS2, and MS3 Docker Build & Environment Verification Passed.

chamalhansika24-a11y / E-Voting-Microservices

Q

Type / to search

<> Code

Issues

Pull requests

▶ Actions

Projects

Wiki

Security and quality

Insights

Settings

Actions

New workflow

All workflows

E-Voting Microservices CI/CD Pipeline

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

All workflows

Showing runs from all workflows

1 workflow run

Workflow ▼

Event ▼

Status ▼

Branch ▼

Actor ▼

✓

[Initial commit with CI/CD Pipeline](#)

main

E-Voting Microservices CI/CD Pipeline #1: Commit f157e16 pushed by chamalhansika24-a11y

2 minutes ago

12s

...

Automated DevSecOps Pipeline (Jenkins CI/CD Verification):

✓ Multi-stage verification passed for SCM Checkout, MS1 (Auth), MS2 (Voting Core), MS3 (Analytics), and Docker Orchestration.

Script ?

```
1 pipeline {
2     agent any
3
4     stages {
5         stage('Checkout SCM') {
6             steps {
7                 echo 'Pulling repository code...'
8                 git branch: 'main', url: 'https://github.com/chamalhansika24-a11y/E-Voting-Microservices.git'
9             }
10        }
11
12        stage('Verify MS1 (Auth)') {
13            steps {
14                echo 'Auditing MS1 User Auth logic...'
15                bat 'dir ms1-user-auth\\ms1-user-auth'
```

Automated DevSecOps Pipeline (Jenkins CI/CD Verification)

✓ Multi-stage verification passed for SCM Checkout, MS1 (Auth), MS2 (Voting Core), MS3 (Analytics), and Docker Orchestration.

 Jenkins

E-Voting-Microservices-Pi...

Status

Changes

Build Now

Configure

Delete Pipeline

Full Stage View

Stages

Rename

Pipeline Syntax

Builds >

Filter

Today

✓ #4 10:49 AM

✓ #3 10:43 AM

✗ #2 10:41 AM

✓ #1 10:35 AM

✓ E-Voting-Microservices-Pipeline

Add desc

Stage View

Average stage times:
(full run time: ~9s)

#4

Sep 14 10:49

No Changes

Checkout SCM	Verify MS1 (Auth)	Build Core (MS2)	Verify Analytics (MS3)	Docker Compose	Post Actions
4s	878ms	590ms	519ms	560ms	180ms
4s	878ms	590ms	519ms	560ms	180ms

Permalinks

- Last build (#4), 56 sec ago
- Last stable build (#4), 56 sec ago
- Last successful build (#4), 56 sec ago
- Last failed build (#2), 8 min 55 sec ago
- Last unsuccessful build (#2), 8 min 55 sec ago
- Last completed build (#4), 56 sec ago

Microservice 3 (MS3) – Real-Time Results & Analytics

- **Direct Blockchain Reading** : Reads live vote tallies directly from the Ethereum Smart Contract without relying on MS1 or MS2.
- **Independent & Decoupled Architecture** : Runs as an isolated service to guarantee that public result tracking remains accessible even if authentication or voting services experience downtime.
- **Real-Time UI Updates** : Dynamically updates vote count cards on the frontend without requiring page refreshes (zero-latency updates).
- **Tamper-Proof Transparency** : Ensures public auditability by fetching immutable result data directly from the decentralized ledger.

Microservice 3 (MS3) – Real-Time Results & Analytics

Technologies Used (Tech Stack)

- **Backend Runtime** : Node.js, Express.js
- **Blockchain Integration** : Web3.js / Ethers.js (Smart Contract Event Listener & Call Read Methods)
- **Frontend UI & Data Viz** : HTML5, CSS3, JavaScript (DOM Manipulation / Event Listener)
- **Containerization** : Docker & Docker Compose (Isolated Port 5003)

End-to-End System Execution Workflow

Step 1: Election Initialization (MS1)

- Admin sets up election details (title, candidates, duration) and uploads the voter CSV file.

Step 2: Automated Email Dispatch (MS1)

- MS1 parses the CSV file and sends unique, secure voting links to registered voters via Nodemailer.

E-Voting Admin Dashboard

1. Schedule Election

Election Title

Annual Election 2026

Question

Select Candidate

Candidates (Comma separated)

Candidate A, Candidate B, Candidate C

Start Time

09/13/2026 11:56 PM

End Time

09/14/2026 11:56 PM

Schedule Election

2. Voter Registration (CSV Upload)

Note: The CSV file MUST contain a column named **email**.

Choose File

email,name,voter_id.csv

Upload & Send Emails

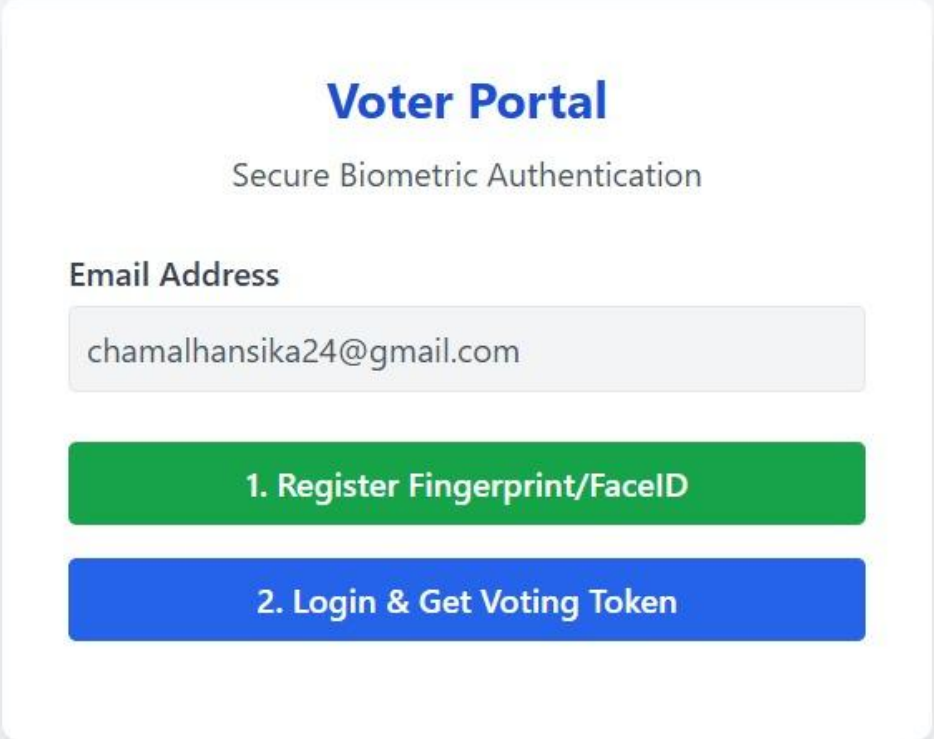
End-to-End System Execution Workflow

Step 3: Biometric Credential Registration (MS1)

- Voter clicks the email link and completes hardware biometric registration (Fingerprint/FaceID) via WebAuthn Protocol.

Step 4: Authentication & JWT Issuance (MS1)

- Voter logs in using biometrics; MS1 validates identity and issues a cryptographically signed JWT Token.



The image shows a 'Voter Portal' interface for 'Secure Biometric Authentication'. It includes an 'Email Address' field with the value 'chamalhansika24@gmail.com'. Below the field are two buttons: a green button labeled '1. Register Fingerprint/FaceID' and a blue button labeled '2. Login & Get Voting Token'.

Voter Portal

Secure Biometric Authentication

Email Address

chamalhansika24@gmail.com

1. Register Fingerprint/FaceID

2. Login & Get Voting Token

End-to-End System Execution Workflow

Voter Portal

Secure Biometric Authentication

Email Address

1. Register Fingerprint/FaceID

2. Login & Get Voting Token



Voter Portal

Secure Biometric Authentication

Email Address

1. Register Fingerprint/FaceID

2. Login & Get Voting Token

Registration Successful! You can now login.



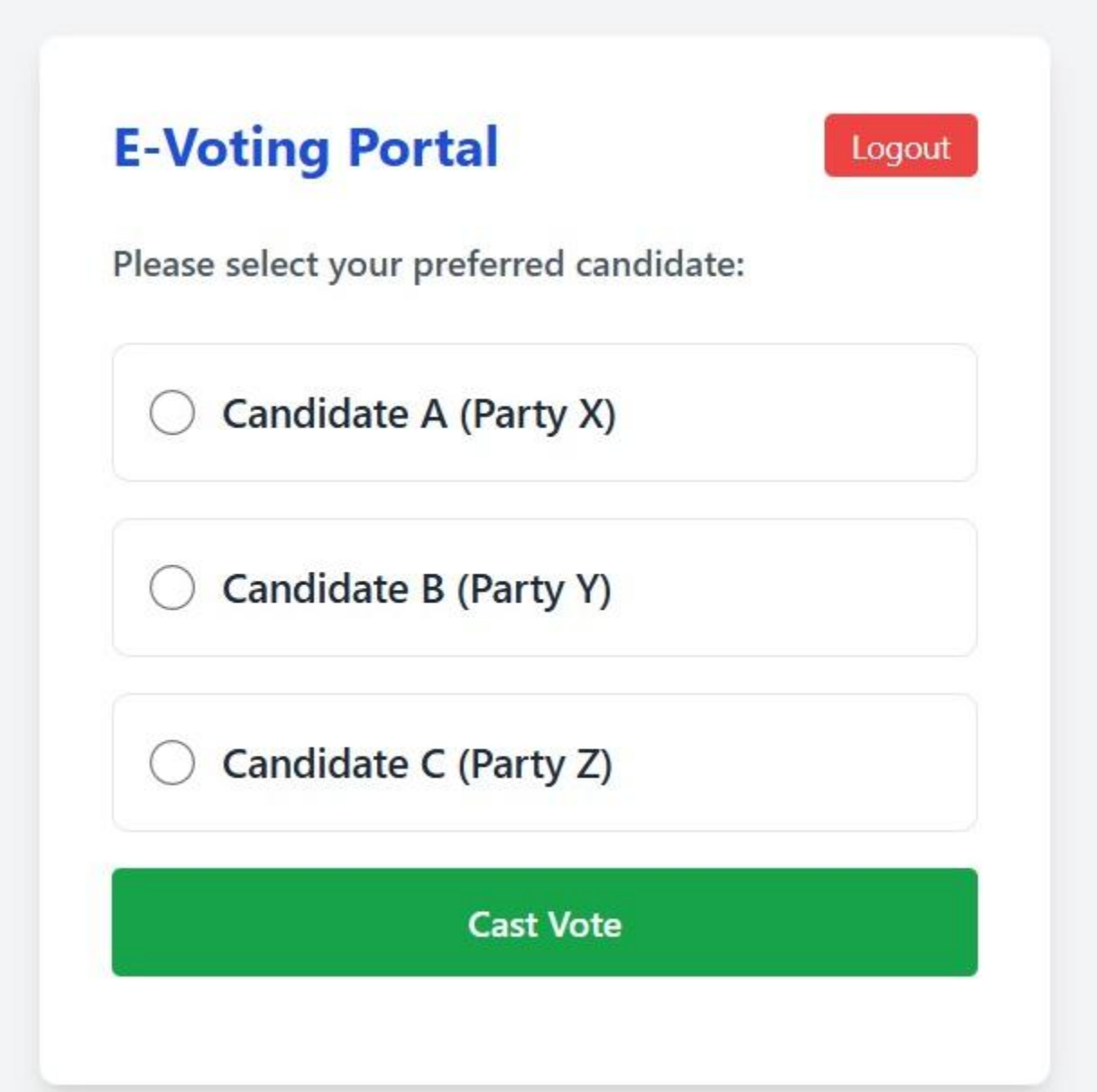
End-to-End System Execution Workflow

Step 5: Vote Submission & Token Transfer (MS1 → MS2)

- Voter selects a candidate on vote.html. The frontend attaches the JWT Token and submits the vote request to MS2 via Axios HTTP API.

Step 6: Token Verification & Smart Contract Execution (MS2)

- MS2 verifies the JWT signature and executes the Solidity Smart Contract using Web3.py on the Ethereum Network (Ganache).



The image shows a mockup of an E-Voting Portal. At the top left, the text "E-Voting Portal" is displayed in blue. At the top right, there is a red "Logout" button. Below the header, the text "Please select your preferred candidate:" is shown. There are three radio button options, each in a white box with a light gray border: "Candidate A (Party X)", "Candidate B (Party Y)", and "Candidate C (Party Z)". At the bottom, there is a large green button labeled "Cast Vote".

End-to-End System Execution Workflow

E-Voting Portal

[Logout](#)

Please select your preferred candidate:

☐ Candidate A (Party X)

☐ Candidate B (Party Y)

☐ Candidate C (Party Z)

Cast Vote



E-Voting Portal

[Logout](#)

Please select your preferred candidate:

☐ Candidate A (Party X)

☐ Candidate B (Party Y)

☒ Candidate C (Party Z)

Cast Vote

Vote Cast Successfully! Thank you.

End-to-End System Execution Workflow

Step 7: Immutability & Double-Voting Check (MS2 / Blockchain)

- Blockchain records the vote block permanently and updates voter state on-chain to block any secondary voting attempts.

Step 8: Real-Time Public Audit & Result Update (MS3)

- MS3 fetches updated block data directly from the Smart Contract and updates the Live Results UI instantly without page refresh.

Live Election Results

Real-time Blockchain Analytics (Microservice 3)

Candidate_C

1

Total Votes

A decorative graphic on the left side of the slide, consisting of a cluster of hexagons in various shades of blue, orange, and white, arranged in a honeycomb-like pattern. The hexagons are of different sizes and are some are solid colors while others are outlined.

Thank you